Report contains answers to Homework 1, Questions additional notes / documentation about the updated shell interface (with history) will be added to the documentation folder.

English enhanced with AI

XV6 `Shell` implementation

XV6's shell is a user-space program implemented through the use various functions such as `gets` from ulib.c and the system calls read, exec, fork themselves. Source code shows a interesting defined function `getcmd` which relies on `gets` (Canonical Mode) which fills up a buffer on \n or EOF characters. This later proved to be a problem when implementing arrow-up and arrow-down keys for shell history access. Shell originates in `main()` where it sets up standard file descriptors by opening "console" three times to ensure stdin, stdout, and stderr are properly initialized. The shell then enters an infinite loop calling `getcmd()` to read user input and `fork()` to execute commands.

The shell's command parsing is straightforward. It tokenizes input using whitespace delimiters and handles basic redirection operators like `>` and `<`. The `runcmd()` function recursively processes parsed command structures, supporting pipes through `pipe()` system calls and I/O redirection through `dup()` and `close()`.

The shell's error handling is minimal, often just calling `panic()` or `exit(1)` on failures. This reflects XV6's teaching-focused design where robustness takes a backseat to clarity and simplicity.

Process creation

Process creation varies widely between Window's Server / API driven architecture vs Linux and XV6's system call interface. XV6, and Linux follow a common architecture pattern where a process is cloned / forked and then a system call for execution is called. While the actual call's and implementations defer the essence remains the same between them.

(Research not verified) On Linux e.g. on a GUI system with GNOME interface when a user clicks the icon of a program e.g. Chrome, a command sequence executes the command is first passed to GNOME shell running on top of Wayland, icon click will be resolved into a command like `gio launch google-chrome.desktop` where `.desktop` is application launchers file extension for GNOME to read.

> `.desktop` file describes to GNOME how to run the program. It may e.g. point to /usr/bin/google-chrome-stable.

Then GNOME shell calls `clone()` system call followed by `execve("/usr/bin/google-chrome-stable", …)` to replace the child's memory with Chrome's executable.

Process is theoretically same for XV6, when a user tries to launch a program, XV6's shell creates a copy of itself using the `fork()` system call and then uses `exec()` call to replace current process's memory with the program. The process of creating new processes may seem similar but underlying complexity is very high in Linux compared to XV6.

(Personal observation) XV6's `fork()` implementation is elegantly simple compared to Linux's `clone()`. In XV6, `fork()` literally copies the entire process address space using `uvmcopy()`, while Linux's `clone()` supports flags like `CLONE_VM` to share memory between parent and child, enabling threads and other advanced features.

The XV6 kernel's `fork()` system call in `proc.c` allocates a new process structure, copies the parent's page table, and sets up the child to return 0 while parent gets the child's PID. This clean separation makes XV6's process model much easier to understand than Linux's complex task_struct and copy-on-write optimizations.

Windows on the other hand deals with process creation completely differently. Windows API exposes `CreateProcess` in `windows.h` and a underlying API function:

```
BOOL CreateProcessW(
    LPCWSTR lpApplicationName,  // const wchar_t*
    LPWSTR  lpCommandLine,      // wchar_t*
    ...
);
```

This "Creates a new process and its primary thread. The new process runs in the security context of the calling process." - Window's API documentation

This approach seems simpler where no separate `fork` + `exec` calls happen, `CreateProcess()` bundles process creation and program loading into one function call.

After this the Kernal creates a new process object, virtual address space, loads Windows DLL's and more before loading `chrome.exe`'s source into memory PE format. Chrome's execution would start from Windows runtime code and eventually get to `main()`.

(Additional Notes) The Windows approach actually mirrors what happens internally in many Unix systems when you call `posix_spawn()` it's essentially a combined fork+exec operation that's more efficient than the traditional two-step process. However, the Unix philosophy of "do one thing well" means `fork()` and `exec()` remain separate, composable primitives.

There is a fundamental contrast in both OS's philosophy, where there's no concept of a process being a child of another process at launch, instead a `new` process is created out of thin air, and

allocated resources and run. While Unix approach seems complicated, it allows these systems to have goodies like daemons while also having copy-on-write like patterns.

Daemon's can be easily created using double `fork()` calls from the Shell / Terminal, the process can now run in the background silent and isolated.

(Appendix I) Daemon's can be easily created using double `fork()` calls from the Shell / Terminal, the process can now run in the background silent and isolated. First `fork()` detaches from parent, parent exits, first child becomes orphan adopted by init. Second `fork()` ensures the daemon can never acquire a controlling terminal since only session leaders can do that, and the second child is guaranteed not to be a session leader.

XV6 doesn't implement sessions or process groups, so daemon creation is much simpler - just `fork()` and have parent exit. This simplification removes job control capabilities but makes the process model much more tractable for learning.

Regards,
Kabeer

kabeer@otherdev.com